ОТЧЁТ О ПРОДЕЛАННОЙ РАБОТЕ (СРСП) · ЧАСТЬ 2 ПРОЕКТА

Завершение проекта: реализация backend-сервиса маркетплейса с фулфилментом и Flash Sales в Docker

Отчёт о реализации второй части проекта по системному дизайну: архитектура контейнеров (Docker + docker-compose), реализация схемы базы данных PostgreSQL по ER-диаграмме, описание REST API (OpenAPI/Swagger), реализация узла шардированных бакетов остатков с TTL, имитация платёжного шлюза, результаты прогона интеграционных тестов pytest и записка о соответствии требованиям первой части.

1. Цель работы и выбранный технологический стек

Связь с первой частью проекта. В первой части проекта была спроектирована архитектура маркетплейса с собственным фулфилментом, доставкой последней мили и механикой горячих распродаж (Flash Sales). Во второй части (СРСП) спроектированная система реализована в виде работающего многоконтейнерного приложения в Docker Compose с реляционной базой данных, кэшем резервов, внешним Mock-сервисом оплаты и автоматическими интеграционными тестами.

Компонент системы	Выбранная технология	Обоснование выбора
Backend-сервис и API	Python 3.12 + FastAPI + Uvicorn	Высокая производительность, строгая валидация типов через Pydantic v2 и автоматическая генерация спецификации OpenAPI 3.1 / Swagger UI (/docs).
Реляционная СУБД	PostgreSQL 16 + SQLAlchemy 2.0	Полная поддержка ACID-транзакций, внешних ключей (FK), составных индексов и CHECK-ограничений (available_qty >= 0) для защиты от оверселлинга.
In-Memory контур резерва	Redis 7 + FlashSaleReservationEngine	Хранение 10 шардированных бакетов остатков (sku:{id}:bucket:{0..9}) для атомарного резерва стока в момент Flash Sale с TTL = 10 минут.
Mock внешнего сервиса	FastAPI Mock Acquiring Service (порт 8001)	Имитация внешнего банка-эквайера и ОФД онлайн-кассы (54-ФЗ) с проверкой ключей идемпотентности (Idempotency-Key).
Контейнеризация	Docker + Docker Compose	Изолированный запуск всех 4 сервисов (backend, postgres, redis, payment-mock) одной командой с проверками готовности (healthcheck).
Тестирование и Seed	Pytest + FastAPI TestClient + Seed-скрипт	Автоматическое наполнение БД тестовыми данными при старте и сквозные интеграционные тесты всех 15 функциональных требований.

2. Реализация базы данных (ER-диаграмма) и миграций

Миграции и начальное наполнение (Seed). Схема базы данных реализована в двух синхронных представлениях: в виде чистого SQL-скрипта миграции migrations/001_init_schema.sql (автоматически монтируется в /docker-entrypoint-initdb.d/ контейнера PostgreSQL) и в виде декларативных моделей SQLAlchemy в модуле app/models.py. При старте сервиса скрипт app/seed.py автоматически создаёт тестовых покупателей, продавцов, склады фулфилмента, курьеров и товары.

Таблица в PostgreSQL	Первичный и внешние ключи	Связь с ER-диаграммой 1-й части и индексы
users	PK: id	Сущность Покупатель (User). Уникальные индексы по phone и email.
sellers	PK: id	Сущность Продавец (Seller). Хранит ИНН (UNIQUE), рейтинг и процент комиссии площадки.
warehouses	PK: id	Сущность Склад фулфилмента (Warehouse). Регион,

		координаты и тип (FULFILLMENT / DARKSTORE).
couriers	PK: id	Сущность Курьер (Courier). Индекс (shift_status, current_zone) + текущие GPS-координаты.
products	PK: id | FK: seller_id → sellers.id	Сущность Товар / SKU (Product). Поля base_price, promo_price, is_flash_sale, flash_sale_limit.
warehouse_stocks	PK: (warehouse_id, product_id) | FK: warehouse_id, product_id	Промежуточная таблица N:M (Склад ↔ Товар). Хранит адресную ячейку bin_location, available_qty (CHECK >= 0), reserved_qty и версию строки.
orders	PK: id | FK: buyer_id, warehouse_id, courier_id	Центральная сущность Заказ (Order). Связывает покупателя, склад, курьера, позиции и оплату. Индексы (buyer_id, created_at) и (warehouse_id, status).
order_items	PK: id | FK: order_id, product_id, seller_id	Промежуточная таблица N:M (Заказ ↔ Товар). Фиксирует историческую цену покупки price_at_purchase.
payments	PK: id | FK: order_id → orders.id	Сущность Платёж (Payment). Содержит UNIQUE индекс по idempotency_key для защиты от двойного списания.
outbox_events, cart_items, reviews	PK: id | FK к соответствующим сущностям	Таблицы паттерна Transactional Outbox (события шины Saga), корзины покупателя и отзывов о товаре.

3. Реализация REST API и бизнес-логики (ФТ-1 .. ФТ-15)

В модуле app/main.py реализованы все обязательные эндпоинты для четырёх ролей пользователей, а также сложный узел из Этапа 5 первой части (модуль app/flash_sale_engine.py): разбиение остатка акционного товара на 10 независимых бакетов sku:{id}:bucket:{0..9}, выбор бакета по hash(user_id) % 10, мягкий резерв на 600 секунд (10 минут) и компенсирующий возврат стока при отмене или истечении времени оплаты.

Роль и требование	HTTP-метод и маршрут (Endpoint)	Описание бизнес-логики и валидации
Покупатель (ФТ-1)	GET /api/v1/catalog/products	Поиск товаров по строке запроса, категории, бренду, цене и флагу Flash Sale.
Покупатель (ФТ-2)	GET /api/v1/catalog/products/{id}	Детальная карточка товара: актуальная цена, остатки по складам, 10 бакетов Flash Sale и отзывы.
Покупатель (ФТ-3)	POST /api/v1/cart/items GET /api/v1/cart/{buyer_id}	Добавление товара в корзину, применение промокода (SALE5) и расчёт слотов доставки.
Покупатель (ФТ-4 + Узел Этапа 5)	POST /api/v1/orders	Оформление заказа: атомарный резерв в шардированном бакете + резерв в WarehouseStock с TTL = 10 мин и записью события в Outbox.
Покупатель (ФТ-5)	POST /api/v1/orders/{id}/pay	Онлайн-оплата через Mock-сервис банка с проверкой Idempotency-Key (идемпотентный повтор без двойного списания).
Покупатель (Saga Компенсация)	POST /api/v1/orders/{id}/cancel	Отмена неоплаченного заказа: компенсирующий возврат единиц товара обратно на склад и в бакеты Redis.
Покупатель (ФТ-6)	GET /api/v1/orders/{id}/tracking POST /api/v1/orders/{id}/reviews	Отслеживание текущего статуса заказа и GPS-позиции курьера; публикация отзыва после доставки.
Продавец (ФТ-7, ФТ-8)	POST /api/v1/seller/products PUT /api/v1/seller/products/{id}/flash-sale	Создание карточки SKU и запуск горячей распродажи с автоматической нарезкой стока на 10 бакетов.
Продавец (ФТ-9, ФТ-10)	POST /api/v1/seller/inbound-supplies GET /api/v1/seller/{id}/analytics	Оформление поставки товара на склад и расчёт аналитики продаж (GMV, комиссия маркетплейса, сумма к выплате).

Склад WMS (ФТ-11..ФТ-13)	POST /api/v1/wms/receive GET /api/v1/wms/picking-tasks POST /api/v1/wms/orders/{id}/pack	Адресная приёмка по ячейкам (bin_location), построение отсортированного маршрута сборки и упаковка с генерацией штрихкода PKG-XXXXXX.
Курьер (ФТ-14, ФТ-15)	POST /api/v1/delivery/orders/{id}/pickup POST /api/v1/delivery/couriers/{id}/location POST /api/v1/delivery/orders/{id}/complete	Приёмка заказа курьером со склада, отправка GPS-координат в реальном времени и подтверждение выдачи проверочным кодом клиента.

4. Контейнеризация (Docker + docker-compose) и проверка работы

В корне проекта расположены Dockerfile основного сервиса, mock_payment/Dockerfile для имитации банка-эквайера и манифест docker-compose.yml, связывающий 4 контейнера (postgres, redis, payment-mock, backend) в единую сеть с проверками готовности (service_healthy).

```
# 1. Запуск всей инфраструктуры в Docker Compose:
cd marketplace-service
docker compose up --build -d

# 2. Открытие интерактивной документации Swagger UI:
# http://localhost:8000/docs

# 3. Запуск интеграционных автотестов pytest (5 комплексных тестовых сценариев):
pytest -v
# Результат прогона:
# tests/test_marketplace.py::test_01_health_and_observability_headers PASSED [ 20%]
# tests/test_marketplace.py::test_02_catalog_search_and_product_card_ft1_ft2 PASSED [ 40%]
# tests/test_marketplace.py::test_03_cart_with_promo_code_ft3 PASSED [ 60%]
# tests/test_marketplace.py::test_04_full_order_lifecycle_ft4_to_ft15 PASSED [ 80%]
# tests/test_marketplace.py::test_05_seller_cabinet_and_flash_sale_anti_overselling_stage5 PASSED [100%]
# ======================== 5 passed in 4.53s ========================
```

5. Записка о соответствии требованиям первой части проекта

Ниже приведена сводная матрица соответствия реализованного сервиса требованиям, сформулированным в первой части проекта (системном дизайне маркетплейса), включая пояснения по архитектурным компромиссам локальной Docker-сборки.

Раздел 1-й части проекта	Статус реализации	Подробности реализации и принятые инженерные решения
Функциональные требования (ФТ-1 .. ФТ-15, все 4 роли)	Реализовано полностью (15 из 15)	Реализованы все 15 пользовательских сценариев для Покупателя, Продавца, Сотрудника склада фулфилмента (WMS) и Курьера доставки. Все маршруты задокументированы в OpenAPI (Swagger).
Нефункциональные требования (NFR: латентность, идемпотентность, логи, конфиг)	Реализовано полностью	• Конфигурируемость: все параметры вынесены в переменные окружения (app/config.py, .env.example). • Наблюдаемость и скорость: middleware логирует каждый запрос и возвращает заголовок X-Process-Time-Ms (< 15 мс при требовании < 200 мс). • Rate Limiting: защита от перегрузки (HTTP 429). • Идемпотентность оплат: UNIQUE индекс по idempotency_key.
Глубокий разбор сложного узла (Этап 5: Резерв стока в Flash Sale)	Реализовано полностью + покрыто тестом	Реализован класс FlashSaleReservationEngine: деление акционного стока на 10 шардированных бакетов (sku:{id}:bucket:{0..9}), маршрутизация по hash(user_id) % 10, мягкий резерв с TTL = 600 сек и компенсационный возврат при отмене. Автотест подтверждает 0% оверселлинга.
Схема базы данных и ER-	Реализовано	Реализованы все сущности ER-диаграммы во главе с

диаграмма (Этап 6)	полностью (100% совпадение)	центральной таблицей orders и промежуточными таблицами развязки связей N:M (order_items и warehouse_stocks), включая все PK, FK, CHECK-ограничения и индексы.
Внешние интеграции и асинхронная шина событий (Этап 4 и Этап 7)	Реализовано через Mock-контейнер и Transactional Outbox	• Внешний банк-эквайер вынесен в отдельный Docker-контейнер payment-mock (порт 8001) с Circuit Breaker fallback. • Асинхронная шина событий Saga реализована через таблицу outbox_events (паттерн Transactional Outbox), что позволяет запускать проект на одном ноутбуке без тяжёлого кластера из 64 серверов.